Objektorientierte Programmierung

WOCHE 3: DESIGN PATTERNS II – VERHALTEN UND INTERAKTION

PETER BERNHARDT

März 2026

1

Teil 1: Einführung in Verhaltensmuster

2

Willkommen zu Woche 3

In der letzten Woche haben wir gelernt, wie man Objekte sauber erzeugt (Factory) und strukturiert (Adapter).

Heute widmen wir uns der dritten und oft wichtigsten Kategorie von Design Patterns: den **Verhaltensmustern (Behavioral Patterns)**.

Lernziele:

- Strategy-Pattern für flexible Algorithmen anwenden.
- Observer-Pattern für ereignisgesteuerte Systeme nutzen.
- State-Pattern zur Abbildung komplexer Workflows implementieren.

3

Was sind Verhaltensmuster?

Während Strukturmuster beschreiben,

wie Objekte zusammengebaut werden,

definieren Verhaltensmuster,

wie Objekte miteinander kommunizieren und wie Aufgaben zwischen ihnen verteilt werden.

Sie helfen uns, komplexe Kontrollflüsse übersichtlich und entkoppelt zu gestalten.

4

Bedeutung für die öffentliche Verwaltung

Verwaltungsprozesse sind hochgradig regelbasiert und zustandsbehaftet.

Regeln ändern sich: Gebührensatzungen oder Gesetze werden modifiziert.

Ereignisse passieren: Ein Antragsteller muss benachrichtigt werden, wenn ein Dokument fehlt.

Zustände wechseln: Ein Bauantrag durchläuft Stufen von “Eingereicht” bis “Genehmigt”.

5

Fokus dieser Woche

Wir konzentrieren uns auf drei essenzielle Muster:

1. **Strategy:** Algorithmen austauschen (z.B. Steuerberechnungen).
2. **Observer:** Systeme entkoppelt informieren (z.B. Status-Emails).
3. **State:** Komplexe Workflows steuern (z.B. Antragsprozesse).

6

Teil 2: Das Strategy Pattern

7

Einführung: Strategy Pattern

Das Strategy-Pattern definiert eine Familie von Algorithmen, kapselt jeden einzelnen in einer eigenen Klasse und macht sie untereinander austauschbar.

Dadurch kann der Algorithmus unabhängig vom Klienten verändert werden.

8

Das Problem: Endlose If-Else-Ketten

Stellen Sie sich Code vor, der je nach Kommune eine andere Gebühr berechnet:

```
if (stadt == "Berlin") { berechneBerlinGebuehr(); }
else if (stadt == "Muenchen") { berechneMuenchenGebuehr(); }
else if (stadt == "Hamburg") { ... }
```

Dieser Code verstößt gegen das **Open-Closed-Prinzip**.

Bei jeder neuen Stadt muss die zentrale Berechnungs-Klasse modifiziert werden.

9

Anwendung: Flexible Gebührenberechnung

In einem landesweiten System zur Berechnung von Hundesteuern hat jede Kommune ihre eigene Satzung.

Wir lagern diese spezifische Logik in einzelne Strategie-Klassen aus, die alle dasselbe Interface bedienen.

10

Strategy: Das Interface (Code)

Wir definieren einen gemeinsamen “Vertrag” für alle Satzungen.

```
// Das Interface für alle Strategien
public interface IGebuehrenStrategie
{
    double Berechne(double basisBetrag);
}
```

Egal wie komplex die interne Berechnung einer Stadt ist, nach außen gibt sie nur das Endergebnis zurück.

11

Strategy: Konkrete Strategien (Code)

Jede Kommune bekommt ihre eigene Klasse.

```
// Konkrete Strategie (Satzung Berlin)
public class SatzungBerlin : IGebuehrenStrategie
{
    public double Berechne(double basis)
        => basis * 1.05; // 5% Verwaltungskostenzuschlag
}

// Konkrete Strategie (Satzung München)
public class SatzungMuenchen : IGebuehrenStrategie
{
    public double Berechne(double basis)
        => basis + 15.00; // Pauschalgebühr
}
```

12

Strategy: Der Kontext (Code)

Die Kern-Anwendung (`GebuehrenRechner`) kennt die genauen Formeln gar nicht.

```
// Der Kontext
public class GebuehrenRechner
{
    private IGebuehrenStrategie strategie;

    // Strategie kann zur Laufzeit getauscht werden
    public void SetStrategie(IGebuehrenStrategie strategie)
    {
        this.strategie = strategie;
    }

    public double BerechneEndbetrag(double basis)
    {
        if (strategie == null)
            throw new InvalidOperationException(
                                        "Keine Satzung gewählt!");

        return strategie.Berechne(basis);
    }
}
```

13

Vorteile des Strategy Patterns

Extreme Flexibilität: Sie können zur Laufzeit (z.B. basierend auf einer Benutzereingabe) das Verhalten des Programms ändern.

Open-Closed Principle: Um Leipzig hinzuzufügen, schreiben Sie einfach die Klasse `SatzungLeipzig`. Der Code im `GebuehrenRechner` bleibt unangetastet.

Testbarkeit: Jede Satzung kann unabhängig als kleiner Unit-Test geprüft werden.

14

Teil 3: Das Observer Pattern

15

Einführung: Observer Pattern

Das Observer-Pattern (Beobachter-Muster) definiert eine 1:n-Abhängigkeit zwischen Objekten.

Wenn sich der Zustand des “Subjekts” ändert, werden alle abhängigen “Beobachter” automatisch benachrichtigt.

16

Das Problem: Polling vs. Push

Polling (Schlecht):

"Ist der Antrag fertig? Nein. Ist er jetzt fertig? Nein."
Die ständigen Nachfragen belasten das System.

Push / Observer (Gut):

"Sag mir Bescheid, sobald der Antrag fertig ist. Ich lege mich solange schlafen."
Das Observer-Pattern ermöglicht ereignisgesteuerte (event-driven) Architekturen.

17

Anwendung: Statusänderungen im Bürgerportal

Wenn ein Sachbearbeiter einen Antrag auf "Genehmigt" setzt, müssen mehrere unabhängige Systeme reagieren:

1. Das Bürger-Dashboard im Frontend.
2. Der E-Mail-Server, der eine Bestätigung sendet.
3. Das Archivsystem, das den Vorgang wegschreibt.
   Es wäre fatal, wenn die `Antrag`-Klasse den E-Mail-Server direkt aufrufen müsste (enge Kopplung).

18

Observer: Das Beobachter-Interface (Code)

Jedes System, das über Änderungen informiert werden will, muss dieses Interface implementieren.

```
// Das Interface für die Beobachter
public interface IStatusBeobachter
{
    void Aktualisieren(string neuerStatus);
}
```

19

Observer: Das Subjekt / Der Antrag (Code 1/2)

Der Antrag verwaltet eine Liste seiner Beobachter.

```
// Das Subjekt, das beobachtet wird
public class Antrag
{
    private readonly List<IStatusBeobachter> _beobachter =
                                new List<IStatusBeobachter>();

    public string Status { get; private set; }

    public void RegistriereBeobachter(IStatusBeobachter b)
        => _beobachter.Add(b);

    public void EntferneBeobachter(IStatusBeobachter b)
        => _beobachter.Remove(b);

// ...
```

20

Observer: Benachrichtigungslogik (Code 2/2)

Sobald der Zustand sich ändert, werden alle informiert.

```
// ... Fortsetzung der Klasse Antrag

    public void SetzeStatus(string neuerStatus)
    {
        Status = neuerStatus;
        BenachrichtigeAlle();
    }

    private void BenachrichtigeAlle()
    {
        foreach (var b in _beobachter)
        {
            b.Aktualisieren(Status);
        }
    }
}
```

21

Observer: Der konkrete Beobachter (Code)

Der E-Mail Service versendet Benachrichtigungen bei Statusänderungen.

```
// Konkreter Beobachter: E-Mail Dienst
public class EmailService : IStatusBeobachter
{
    public void Aktualisieren(string neuerStatus)
    {
        Console.WriteLine(
            $"Sende E-Mail: Ihr Antrag ist nun '{neuerStatus}'.");

        // E-Mail Sende-Logik hier...
    }
}
```

22

Observer: Das Hauptprogramm (Code)

Der E-Mail Service registriert sich beim Antrag und reagiert auf Änderungen.

```
var antrag = new Antrag(); // Subjekt

var emailService = new EmailService(); // Beobachter

antrag.RegistriereBeobachter(emailService); // Registrierung

Console.WriteLine("Statuswechsel: Genehmigt");
antrag.SetzeStatus("Genehmigt"); // Benachrichtigung erfolgt

Console.WriteLine("E-Mail-Service wird abgemeldet...");
antrag.EntferneBeobachter(emailService);

Console.WriteLine("Statuswechsel: Archiviert");
antrag.SetzeStatus("Archiviert"); // keine Benachrichtigung mehr
```

23

Vorteile des Observer Patterns

Lose Kopplung: Der `Antrag` kennt seine Beobachter nicht im Detail, er weiß nur, dass sie das Interface implementieren.

Dynamik: Beobachter können zur Laufzeit hinzugefügt oder entfernt werden.

Anmerkung: In modernem C# wird das Observer-Pattern oft nativ durch `Events` und `Delegates` abgebildet. Das Prinzip dahinter ist jedoch identisch!

24

Teil 4: Das State Pattern

25

Einführung: State Pattern

Das State-Pattern erlaubt es einem Objekt, sein Verhalten grundlegend zu ändern, wenn sich sein interner Zustand ändert.

Es kapselt zustandsspezifisches Verhalten in eigenen Klassen.

26

Das Problem der Zustandsverwaltung

Wie programmieren Sie einen Antrag, der verschiedene Phasen durchläuft?

Meist endet es in unleserlichen Switch-Statements:

```
public void Genehmigen() {
    if (status == "Eingereicht") { throw error; }
    else if (status == "Geprüft") { status = "Genehmigt"; }
    else if (status == "Genehmigt") { throw error; }
}
```

Das skaliert nicht bei komplexen Workflows mit vielen Regeln.

27

Anwendung: Bauantrags-Workflow

Ein Bauantrag hat einen strikten Lebenszyklus:

1. Eingegangen
2. In Prüfung (Unterlagen können nachgefordert werden)
3. Genehmigt / Abgelehnt

In jedem dieser Zustände sind unterschiedliche Aktionen erlaubt oder verboten.

28

State: Die abstrakte Basisklasse (Code)

Wir definieren eine Klasse, die alle möglichen Aktionen als Methoden vorgibt.

```
public abstract class BauantragStatus
{
    // Das 'kontext' Argument erlaubt es dem Zustand,
    // den Status des Antrags auf den nächsten Schritt zu setzen.
    public abstract void Bearbeiten(Bauantrag kontext);
    public abstract void Genehmigen(Bauantrag kontext);
}
```

29

State: Ein konkreter Zustand (Code)

Wir programmieren die Logik spezifisch für den Zustand "Eingereicht".

```
// Konkreter Zustand: Eingereicht
public class EingereichtStatus : BauantragStatus
{
    public override void Bearbeiten(Bauantrag kontext)
    {
        Console.WriteLine("Prüfung wird gestartet...");

        // Zustandswechsel auslösen!
        kontext.SetzeZustand(new InPruefungStatus());
    }

    public override void Genehmigen(Bauantrag kontext)
    {
        Console.WriteLine(
                    "Fehler: Antrag muss erst geprüft werden!");
    }
}
```

30

State: Ein konkreter Zustand (Code)

Wir programmieren die Logik spezifisch für den Zustand “InPruefung”.

```
// Konkreter Zustand: InPruefung
public class InPruefungStatus : BauantragStatus
{
    public override void Bearbeiten(Bauantrag kontext)
        => Console.WriteLine("Fehler: Prüfung läuft bereits.");


    public override void Genehmigen(Bauantrag kontext)
    {
        Console.WriteLine("Antrag wurde genehmigt.");
        kontext.SetzeZustand(new GenehmigtStatus());
    }
}
```

31

State: Ein konkreter Zustand (Code)

Wir programmieren die Logik spezifisch für den Zustand “Genehmigt”.

```
// Konkreter Zustand: Genehmigt
public class GenehmigtStatus : BauantragStatus
{
    public override void Bearbeiten(Bauantrag kontext)
        => Console.WriteLine(
                "Fehler: Antrag ist bereits abgeschlossen.");


    public override void Genehmigen(Bauantrag kontext)
        => Console.WriteLine(
                    "Fehler: Antrag ist bereits genehmigt.");
}
```

32

State: Die Kontext-Klasse / Der Antrag (Code)

Der Antrag delegiert alle fachlichen Methoden einfach an sein aktuelles Status-Objekt.

```
// Der Kontext
public class Bauantrag
{
    private BauantragStatus _aktuellerStatus;

    // Start-Zustand
    public Bauantrag() => _aktuellerStatus =
                                        new EingereichtStatus();

    public void SetzeZustand(BauantragStatus status)
        => _aktuellerStatus = status;

    // Delegation an den jeweiligen Zustand
    public void Bearbeiten() => _aktuellerStatus.Bearbeiten(this);
    public void Genehmigen() => _aktuellerStatus.Genehmigen(this);
}
```

33

State: Das Hauptprogramm (Code)

Der Antrag durchläuft die einzelnen Status.

```
var antrag = new Bauantrag();

Console.WriteLine("-- Antrag eingereicht --");
antrag.Genehmigen(); // Fehler: noch nicht geprüft

Console.WriteLine("-- Bearbeitung starten --");
antrag.Bearbeiten(); // Zustandswechsel: -> InPruefung

Console.WriteLine("-- Genehmigung erteilen --");
antrag.Genehmigen(); // Zustandswechsel: -> Genehmigt

Console.WriteLine("-- Weiteres Bearbeiten nach Genehmigung --");
antrag.Bearbeiten(); // Fehler: bereits abgeschlossen
```

34

State: Das Hauptprogramm (Code)

Der Antrag durchläuft die einzelnen Status.

```
var antrag = new Bauantrag();

Console.WriteLine("-- Antrag eingereicht --");
antrag.Genehmigen(); // Fehler: noch nicht geprüft

Console.WriteLine("
antrag.Bearbeiten()

Console.WriteLine("
antrag.Genehmigen()

Console.WriteLine("
antrag.Bearbeiten()
```

```
-- Antrag eingereicht --
Fehler: Antrag muss erst geprüft werden!

-- Bearbeitung starten --
Prüfung wird gestartet...

-- Genehmigung erteilen --
Antrag wurde genehmigt.

-- Weiteres Bearbeiten nach Genehmigung --
Fehler: Antrag ist bereits abgeschlossen.
```

35

Unterschiede: State vs. Strategy

Obwohl die Struktur ähnlich ist, ist die **Absicht** unterschiedlich:

Strategy:
Der Client wählt den Algorithmus einmal aus (z.B. Berlin).

State:
Das Objekt wechselt seinen “Algorithmus” (Zustand) selbstständig über die Zeit (Workflow).

36

Vorteile des State Patterns

- Wir eliminieren gewaltige bedingte Anweisungen (`if/switch`).
- Das Hinzufügen eines neuen Zwischenschritts (z.B. `InWiderspruchsPruefung`) erfordert lediglich eine neue Klasse und eine Anpassung des Vorgängers.
- Der Code für einen bestimmten Zustand ist an einem einzigen Ort versammelt (Single Responsibility).

37

Teil 5: Zusammenfassung

38

Wrap-up Woche 3

Strategy:
Kapselt Algorithmen (z.B. verschiedene Steuergesetze) und macht sie austauschbar, ohne die Kern-Anwendung zu ändern.

Observer:
Informiert viele Interessenten entkoppelt über Zustandsänderungen (Publish/Subscribe-Prinzip).

State:
Objektorientierte Alternative zur Switch-Case-Hölle für komplexe Lebenszyklen und Workflows.

39

Muster kombinieren

In realen Anwendungen treten Muster selten isoliert auf.

Ein `Bauantrag` (State) könnte eine `GebuehrenStrategie` (Strategy) nutzen, und bei jedem Zustandswechsel seine `IStatusBeobachter` (Observer) informieren.

40

Die 3 Takeaways für heute

1. Strategy für austauschbare Regeln.
2. Observer für entkoppelte Benachrichtigungen.
3. State für saubere Workflows.

41

Ausblick auf Woche 4

Nächste Woche verlassen wir die Ebene der Klassenmuster und betrachten die makroskopische Ebene:

Die **SOLID-Prinzipien** und die **Schichtenarchitektur**.

42

Übungsvorbereitung

In der heutigen Übung werden wir ein Workflow-System für Parkausweise implementieren.

Denken Sie daran: Erst den Prozess (Zustände) skizzieren, dann coden!

43